

IN1030

Systemer, krav og konsekvenser

Våren 2026

Obligatorisk oppgave 4

Foranalyse og kravhåndtering

Gruppearbeid

Gruppemedlemmer:

Bruker	Personnavn
justassi	Justas Simutis
daniv	Delfine Vågenes
ninmar	Nina Marthinsen
sondrso	Sondre Solheim

Oppgave 1 – Bakgrunn

Deloppgave A (daniv)

Vi velger ut noen grunner som er relevant til SmartNest.

Noen fordeler ved å utvikle et nytt system:

Egendefinerbar

Du har muligheten til å tilpasse systemet som du ønsker. Dette kan gjøre det lettere å kun møte de kravene som er ønsket av systemet og ikke mer. Dette gjelder for både funksjonelle og ikke-funksjonelle krav, hvor for eksempel du kan få en bedre ytelse ved å spesialisere systemet.

Sikkerhet

Når du ikke er avhengig av tredjepartsbiblioteker eller programvare kan du selv garantere at behandlingen av data skjer på et lovlig vis, og minimerer risikoen for datainnbrudd. Dette er særdeles viktig når du oppbevarer både fullt navn og hvor de er bosatt.

Lisensiering og støtte

Det er ikke garantert at systemet du utnytter deg av vil ha støtte i all fremtid, men så lenge du bruker systemet kan det godt være at du må enten betale for systemet gjennom et abonnement eller at du kan bli tvunget til å gjøre koden open-source fordi du har brukt et system som bruker en lisens som krever dette. Intet av dette er et problem om du bygger systemet selv.

Noen ulemper ved å utvikle et nytt system:

Mye arbeid på forhånd

Selv om det må brukes tid til å forske på om et gammelt system kan gjenbrukes, er det en betydelig større kostnad å starte fra bunnen av. Da må du gjennom hele programvareutviklingsprosessen fram til det som blir nesten kun opprettholdingsfasen. Dette kan ta ganske lang tid fra du utvikler systemet til det kan gjøres klart for utgivelse, hvor det kan ha kommet et annet system som gjør akkurat det du trenger eller teknologien gått forbi og programmet allerede er utdatert fra utgivelsen.

Testet – både hos brukere og i systemet

Det finnes ikke noe en bruker hater enn å måtte sette seg inn i noe nytt, hvor en minste detalj som er uintuitivt kan skape frustrasjon hos bruker. Samtidig kan det være at et allerede eksisterende system har vært i bruk så lenge at det har gått igjennom en robust testprosess (selv om dette ikke skjer for ofte i virkeligheten), noe som gjør at du kan føle deg tryggere på påliteligheten av systemet og en minimering av defekter. Hvis du lager et nytt system, må du på et uavhengig vis gjøre brukerundersøkelsene og systemtestingen som er nødvendig.

Større team

For større systemer blir det fort nødvendig med flere på teamet til å vedlikeholde systemet, hvor du gjerne anskaffer spesialister. Ved å 1 bruke et allerede eksisterende system, er byrden av å oppdatere og holde dette større systemet ikke ditt ansvar og ofte funnet raskere av bedriften du lisensierer fra. Du trenger så ikke å utvide ditt eget team for å vedlikeholde og kun fokusere på kompatibilitet med systemet etterhvert som du oppdaterer det.

Deloppgave B (daniv)

Det er vanskelig å vite om et eksisterende system eller å lage ditt eget system er det beste. Det finnes mange, veldig mange, open-source systemer på dette allerede som støtter både Philips og annet. Det er ikke et veldig stort system en lager, og det bør derfor vurderes om det tjenes å eventuelt lisensiere fra Philips. Her er det viktig å forhøre seg med Philips: er allerede systemet abstrakt nok til å fungere på hvilken som helst smart-enhet? Hvis ikke, må dette refaktoriseres og restruktureres etter man får lov å bruke systemet. Philips Hue er derimot et av de mest populære appene for smart-enheter, som ikke nødvendigvis trenger å komme fra brukeropplevelsen av appen, men heller at mange smarte enheter er produsert av Philips. Allikevel vet man at systemet har vært testet og endret gjennom brukerundersøkelser. Philips Hue tilbyr også langt flere funksjonaliteter som ikke er en del av kravspesifikasjonene gitt fra den ønskede funksjonaliteten, så dette vil faktisk bety at det er mye mer i systemet som en må forholde seg til enn om man hadde lagd det selv.

Fra denne drøftingen, gitt en antakelse at Philips Hue er det eneste andre systemet tilgjengelig, laget et eget system. Dette er på bakgrunn at funksjonaliteten i Philips Hue er mye større enn det som er nødvendig og det er mistenkelig om Philips Hue er abstrakt nok til å støtte en vilkårlig smart-enhet, som betyr at man må allikevel gjøre en del endringer i det allerede eksisterende systemet.

Oppgave 2 – Interessenter

Deloppgave A (justassi)

En interessent er en gruppe eller et individ som kan både påvirke og bli påvirket av et system, men ikke nødvendigvis på en direkte måte. En aktør er en interessent som samhandler direkte med systemet. Forskjellen mellom disse to er at en interessent ikke nødvendigvis samhandler direkte med systemet, mens en aktør gjør det. Derfor kan vi si at en aktør er en subset av en interessent. Det vil si at alle aktører er en type interessent, men ikke nødvendigvis vice versa.

Som et eksempel kan vi tenke oss en app som er støttet av investorer, samt en person som prøver å hacke seg inn for å få uautorisert tilgang til brukernes personlige opplysninger. Hackeren er en aktør fordi vedkommende samhandler direkte med appen, og er dermed også en interessent. Derimot er investorene interessenter, men ikke aktører, siden de ikke samhandler direkte med appen, selv om de blir påvirket økonomisk av dens suksess

Deloppgave B (justassi)

Systemets utviklere:

Interesse	Ansvarsområde
Få lønn, godt arbeidsmiljø, vise god faglig kompetanse.	Design, implementere, og vedlikeholde SmartNest.

SmartNest sine investorer:

Interesse	Ansvarsområde
Ønsker at SmartNest skal lykkes økonomisk og gi avkastning.	Finansiere systemets utviklere og dets drift.

Enhetsprodusenter:

Interesse	Ansvarsområde
Selge enheter, øke kompatibilitet for å nå ut til en bredere målgruppe	Sørge for at enhetene fungerer som planlagt og kompatible med systemer som f.eks. SmartNest

Applikasjonsadministrator, i.e. en autorisert bruker av admin grensesnittet:

Interesse	Ansvarsområde
Lønn, godt arbeidsmiljø, minimere eget manuelt arbeid.	Overvåke systembruk, administrere brukere og håndtere feil i systemet.

SmartNest sine brukere og gjester til disse brukerne

Interesse	Ansvarsområde
Forenkle livet ved å benytte et sikkert, informativt, effektivt og enkelt system.	Bruke systemet til å få informasjon om og kontrollere enheter

Kriminelle hackere og andre trusselaktører:

Interesse	Ansvarsområde
Utnytte svakheter i systemet for å oppnå egne kriminelle mål, f.eks. låse brukere ute fra egne enheter og kreve betaling *	Ingen ansvarsområde. Hackere representerer en trussel mot systemet.

* Såkalt «ransomware»

Deloppgave C (justassi)

Applikasjonsadministratorer, brukere og hackere er alle aktører blant interessentene nevnt i tabellen i oppgave 2B. Dette er fordi at disse aktørene samhandler direkte med systemet. De bruker SmartNest-systemet (enten bruker- eller admin-grensesnittet) på en eller annen måte, enten for å assistere, kontrollere enhetene, eller utføre en trussel.

Derimot, SmartNest investorer, utviklere, og enhetsprodusenter er ikke aktører. Dette er fordi at ingen av dem samhandler med SmartNest sitt bruker- eller admin-grensesnitt, som er den primære måten å samhandle direkte med systemet på. Likevel regnes de fortsatt som interessenter, fordi de indirekte påvirker SmartNest-systemet, og blir påvirket av det.

Oppgave 3 – Utviklingsprosess

Deloppgave A (sondrso)

Plandrevne modeller:

Plandrevne prosessmodeller søker å optimalisere systemets arkitektur og vedlikeholdbarhet gjennom grundig forarbeid og etterarbeid. Til å begynne med var det disse modellene som preget det meste av systemutvikling, inspirert av arbeidsmetodene som fremdeles råder i ingeniørprosjekter. Deriblant finner vi den såkalte fossefallsmodellen, hvor utviklingsprosessen beskrives som en kaskade av strengt adskilte faser, deriblant kravspesifisering, utforming, implementering og testing, forutgått av forundersøkelser og etterfulgt av drift og vedlikehold. Hvis det oppdages brister eller mangler underveis, må da potensielt hele prosjektet tas tilbake til en tidligere fase, for å sikre at man har kontroll på hvordan alle komponenter og funksjoner vekselvirker med hverandre. Imidlertid har de fleste plandrevne utviklingsprosesser etter hvert blitt inkrementelle, dvs. at de ikke lenger gjennomføres som udelelige helheter, men snarere i mindre og mer praktisk håndtérbare etapper, hver med sin egen faseinndeling.

Hovedfordelen med de plandrevne modellene er at hvis det er mulig på et tidlig stadium å overskue alle de viktigste aspektene ved systemet som skal utvikles, oppnår man stor grad av kontroll med hva som utvikles og hvordan. Dette gjør det mulig å planlegge effektive innfallsvinkler og unngå flaskehalser, og det er mulig å inngå kontrakter med kunder som krever full forhåndsspesifisering av funksjonalitet og kostnader. Sikkerhets- eller samfunnskritiske systemer utvikles nesten alltid slik, og gjerne til og med etter fossefallsmodellen, fordi dette gjør det mulig, til en viss grad, å analysere det systemet for brister og sårbarheter allerede mens det befinner seg på planleggingsstadiet. Ulempen med plandreven utvikling er imidlertid at det er veldig vanskelig senere i prosessen å endre kravspesifikasjonen eller utformingen, være seg pga. endrede ønsker fra oppdragsgiver eller fordi man f.eks. har oppdaget logiske brister. Dette kan også skje med ingeniørprosjekter, men der er endringsmotstand en fysisk, materiell realitet, mens i alle fall når det gjelder programvare, har man en viss frihet til å velge en foretrukket fremgangsmåte.

Smidige modeller:

Smidige prosessmodeller, som alltid er inkrementelle, er av nyere dato, men har vunnet mye terreng de siste tiårene. De kjennetegnes av at de vektlegger selve programmeringen, forstått som produksjon av kode, og søker å minimere hvor mye tid og krefter som går med til planlegging, modellering og dokumentasjon. I stedet setter man i gang med implementeringen basert på kun en i beste fall grovmasket kravspesifikasjon, og lar prosessen ta uforutsette vendinger underveis, i takt med nye eller reviderte krav fra oppdragsgiver. Dette kan innebære et stort antall iterasjoner av utforming, implementering og testing, og minimalt med tid brukes på formelle spesifikasjoner. I løpet av utviklingsprosessen er det ingen som vet nøyaktig hvordan systemet skal se ut til slutt, hvilke funksjoner det vil ha, hvor lang tid det vil ta å ferdigstille det eller hvor mye det vil ende med å koste. Smidig utvikling fokuserer på at man skal omfavne endringer og, særlig i den senere beskrevne tidsboksbaserte tilnærmingen, hyppig levere nye versjoner, for å holde kunden kontinuerlig engasjert.

og medansvarlig.

Poenget med smidig utvikling er altså endringstoleranse. For det første kan man lett kan endre komponenter som ennå ikke er implementert, ettersom de sannsynligvis heller ikke er planlagt, og det ikke finnes noen forhåndsavtalt kravspesifikasjon som endringene kan stride mot. For det andre kan eksisterende komponenter endres uten at det samtidig oppstår behov for å revidere tilhørende dokumentasjon. De smidige modellene ble riktignok ikke foreslått fordi det nødvendigvis er uviktig å vite ting på forhånd, men ut fra en erkjennelse av at fortløpende endring er vanskelig å unngå, samt at det uansett er nesten umulig å planlegge helt riktig og fullstendig fra begynnelsen av. Og mangelen på formell dokumentasjon søkes oppveid gjennom selvdokumenterende kodelast, i form av god navngiving, lokale kommentarer og ryddige algoritmer.

For at smidige prosesser skal fungere, må imidlertid kunden være villig til å involvere seg tett og dynamisk underveis, og betale for anvendt tid snarere enn for et forhåndsavtalt produkt. Dessuten må det, fordi kravspesifikasjonene er grovmaskede og ufullstendige, og hele tiden kan endres, foreligge et godt miljø for uformell kommunikasjon. Hvilket igjen forutsetter at utviklerne samarbeider godt og også får tillatelse og romslighet til å organisere arbeidet sitt selv, uten at ledelsen detaljstyrer. Mens i prosjekter som er veldig store eller har veldig mange interesser, slik at det kreves mye, og gjerne skriftlig, kommunikasjon og avtaler, på tvers av organisasjoner, og kanskje også mellom flere arbeidslag adskilt i tid eller rom, og hvor mange av de menneskelige relasjonene ikke er personlige, kan det være nødvendig med en god del formell spesifisering og dokumentasjon for å muliggjøre samarbeid og kontrakter. I tillegg kan det for kritiske systemer være helt nødvendig med utstrakt grad av forhåndsspesifisering og foranalyse.

Deloppgave B (sondrso)

Flytbaserte modeller:

La oss begynne med flytbaserte modeller, ofte representert ved Kanban-modellen. Disse modellene går ut på at deloppgavene i utviklingsprosjektet prioriteres i den rekkefølgen som virker naturlig ut fra arkitekturen i systemet, og at man ikke begynner på en ny oppgave før inneværende oppgave er fullført. I et større arbeidslag kan flere oppgaver riktignok godt pågå parallelt, men da vel å merke ikke altfor mange av gangen, og fordi hver enkelt oppgave kan ta mange uker eller mer, bør parallelle oppgaver velges slik at man unngår flaskehalser, dvs. at de er minst mulig avhengige av hverandre. For å holde styr på hvilke oppgaver som pågår for øyeblikket, fremdrift og evt. problemer som måtte oppstå, opererer man gjerne med en form for tavle, ofte en fysisk tavle med tusjer og lapper, i Kanban-modellen kalt Kanban-tavle, slik at alle utviklerne til enhver tid lett kan se hva de andre holder på med. Laglederen bruker også denne tavla som utgangspunkt for interne statusmøter, samt for fortløpende dialog med kunden, skjønt man i størst mulig grad søker å skjerme det pågående arbeidet fra utforming og prioriteringen av fremtidige oppgaver.

Flytbaserte prosesser lar hver enkelt utvikler fullføre det vedkommende har påbegynt, og kan slik virke motiverende, ikke minst fordi det ligner på den måten mange foretrekker å arbeide når de jobber med alene med et enmannsprosjekt. Det føles da gjerne tilfredsstillende å bytte oppgave først når man har nådd et naturlig hvileskjær. Dessuten mister man minimalt med tid til forvaltning av

oppgavefordeling og overganger mellom oppgaver, ettersom oppgavene og overgangene mellom dem er så få som mulig. En ulempe er imidlertid at det er lett å gå i den fella at man ender med å sløse bort tid på å kjempe seg gjennom oppgaver som viser seg mer krevende enn antatt, og som man alternativt kunne vurdert, på et tidlig stadium, å avbryte for så enten å bortprioritere eller å forsøke på nytt med en bedre fremgangsmåte. Dessuten mister man ved flytbasert tilnærming noen av de tidligere nevnte fordelene ved hyppig levering, fordi leveringsintervallene er uregelmessige og potensielt ganske lange. Imidlertid kan nye oppgaver legges til når som helst, så lenge det oppstår ledig kapasitet.

Tidsboksbaserte modeller:

Den tidsboksbaserte modellen på sin side, i praksis stort sett den som går under navnet Scrum, går ut på at man jobber i såkalte sprinter, som er definerte tidsintervaller på typisk noen få uker. Hver sprint begynner med et møte hvor man, i samråd med representanten for kunden, velger ut prioriterte oppgaver fra en oppsamlet oppgavebank, kalt "backlog". Prioriteringen tar utgangspunkt både i kundens faktiske behov, hvilke oppgaver som avhenger av hverandre, og anslag over hvor mye tid hver av dem vil kreve, basert på kontinuerlig opparbeidet erfaring innad i prosjektet. Hver dag avholdes et kort statusmøte blant utviklerne, og det benyttes kontinuerlig en fysisk tavle e.l., i Scrum-modellen kalt Scrum-tavle, til fellesnotater om fremgangen og evt. problemer innenfor de forskjellige parallelle oppgavene. Ved utgangen av sprinten møtes utviklerne, sammen med kunden, for evaluering, og evt. ufullendte oppgaver stanses og legges tilbake i oppgavebanken.

Fordelen med tidsboks-tilnærmingen er at man hyppig nullstiller seg og gjør nye prioriteringer basert på en potensielt endret situasjon, i stedet for å bruke opp tid på f.eks. fastlåste problemer. Denne jevnlige kursjusteringen fungerer som en tilnærmet garanti for at arbeidsressursene, i form av tid, krefter og kompetanse, utnyttes optimalt. Dessuten sørger den for at prioriteringen av utviklingsoppgavene hele tiden samsvarer med hva som er nyttigst og viktigst for kunden, som forøvrig ofte vil sette pris på jevn og forutsigbar fremgang, med hyppige anledninger til å komme med innspill. Ulempene ved tidsboksing er imidlertid i stor grad de samme som fordelene ved det flytbaserte, ikke minst at det å stadig måtte avbryte ufullendte oppgaver kan vekke frustrasjon hos utviklerne, som fratas den iboende motivasjonen som ligger i å fullføre ting, noe som også lett kan gå på bekostning av kodestil, og medføre etterslep av teknisk gjeld. I tillegg, særlig i Scrum-modellen konkret, går det med en del tid og på oppgaveforvaltning, bla. fordi man ideelt sett skal ha en egen Scrum-master, en slags arbeidsleder som ikke samtidig er produkteier.

Deloppgave C (sondrso)

Hvor mye endring som må påregnes beror på hva som er kildene til kravene. Siden SmartNest ikke utvikles for en bestemt kunde, men snarere er ment som hyllevare for forbrukerne, finnes det heldigvis ingen kunderepresentant som involverer seg eller kan involveres underveis, og aktivt komme med nye ønsker. Imidlertid må vi regne med at kravene er basert på en analyse av markedet og forbrukernes vaner, samt grensesnittene mot evt. tredjeparts programvare, slik som Philips Hue. Begge deler er faktorer utenfor utviklernes kontroll, og alt som er forbundet med forholdsvis ny teknologi og nyoppståtte trender er tilbøyelig til å forandre seg. Imidlertid er dette et spørsmål om tidsperspektiv, og i utgangspunktet fremstår SmartNest som et system av nokså begrenset størrelse, som, gitt at det skal utvikles av mer enn én eller to personer, trolig lar seg ferdigstille på noen

måneder. Og hvis denne antagelsen er riktig, er det lite sannsynlig at forbrukernes vaner eller tredjepartsleverandørenes programvare endrer seg vesentlig i løpet av den tiden det tar å utvikle SmartNest-appen. Vel å merke må vi riktignok regne med å tilpasse oss endringer underveis i den senere vedlikeholdsfasen.

Det er så klart alltid mulig at et eller annet selskap lanserer en banebrytende ny teknologi som brått og uventet endrer trendene i markedet, men likevel vil det for lang tid fremover være mange forbrukere som har investert i smartenheter med eksisterende teknologi, og som vil være interessert i å bruke disse på en effektiv måte. Av det ovenstående utleder vi at den eneste vesentlige endringskilden vi bør ta hensyn til er ønsker og krav som oppstår internt i egen bedrift. Én intern endringskilde er utviklernes egne tester av systemet, som potensielt kan avdekke behov for endringer i kravene, være seg pga. innbyrdes konflikter eller andre logiske brister, eller fordi enkelte krav viser seg umulige, urealistiske eller altfor tidkrvede å implementere. En annen fare er at ledelsen, som i en liten bedrift ofte sammenfaller med eieren eller eierne, forandrer mening om det ene eller det andre, eller rett og slett aldri helt får bestemt seg. I verste fall er lederen eller eieren impulsiv eller vinglete, og forstyrrer flyten i utviklernes arbeidsprosess.

Deloppgave D (sondrso)

Rundt hvilken prosessmodell som best egner seg for det foreliggende systemet, anser jeg det formålstjenlig, ut fra oppgavens art, å foreløpig ikke ta fullstendig stilling, men heller belyse fordeler og ulemper ved den ene modellen fremfor den andre. Hvis det foreligger nokså varierende feridighetsnivå blant utviklerne, har f.eks. plandrevne modeller den fordel at man da kan velge å bruke de dyktigste medarbeiderne til planlegging og utforming, slik at de mindre erfarne kan jobbe med å implementere detaljene innenfor fastlagte rammer. Dermed håndterer man fint at ikke alle er i stand til å ta jevnbyrdig del i en kontinuerlig drøfting og bearbeidelse av arkitekturen i systemet. Dessuten kan det godt hende at ledelsen, eieren eller eierne har en bestemt forestilling om hvordan det ferdige produktet bør se ut, og ikke ønsker å sette i gang med noen som helst implementering før det er fullstendig utredet at det de ønsker seg lar seg gjøre, og hvordan det skal gjøres. Imidlertid oppfyller en liten bedrift lett forutsetningene for å jobbe etter en smidig modell, hvis man ønsker dette, og ettersom vi antar at de største endringskildene er interne, kan hvilken prosessmodell som er beste egnet til og med avhenge av personlighetene til leder, eier eller sentrale utviklere.

En klassisk smidig prosess er fullt mulig også uten involvering av noen kunde, for å nyte godt av endringstoleransen som følger med, eller hvis ingen i bedriften har kompetanse eller tålmodighet til å planlegge hele systemet på forhånd. Generelt er de smidige modellene best egnet nettopp i små miljøer hvor alle jobber på samme sted og kan kommunisere uformelt seg imellom. Og under forutsetning av at ferdige produktet ikke er forhåndsbestemt av en kontrakt. Smidig utvikling forsetter at alle beslutninger kan tas "innomhus", dvs. enten innad i en bedrift som utvikler hylleware for salg på det åpne markedet, slik som her, eller innad i samarbeidet med en kontinuerlig involvert kunde og laget av konsulenter som utvikler på mandat fra denne kunden. Derunder står valget mellom flytbasert tilnærming (Kanban) eller tidsboksbasert tilnærming (Scrum). Begge deler er fullt mulige, men det er verdt å bemerke at tidsboksing medfører en del ekstra rolleapparat og oppgaveforvaltning, og i tillegg kan virke stressende på den enkelte utvikler, og alt dette er friksjon som potensielt går på bekostning av effektiv fremdrift, all den tid det skal håndteres innenfor et lag bestående av kun noen få utviklere.

Oppgave 4 – Kravspesifikasjon

Deloppgave A (ninmar)

Prioritert liste av funksjonelle krav som brukshistorier:

1

Som system ønsker jeg å lagre brukerdata sikkert, slik at informasjon ikke går tapt.

2

Som bruker ønsker jeg å logge inn i appen, slik at jeg får tilgang til mine enheter og innstillinger.

3

Som bruker ønsker jeg å se en oversikt over alle mine smarte enheter, slik at jeg har kontroll over hjemmet mitt.

4

Som bruker ønsker jeg å kunne legge til og fjerne enheter, slik at jeg kan tilpasse systemet til mitt hjem.

5

Som bruker ønsker jeg å se energiforbruk for enhetene mine, slik at jeg kan spare strøm og penger.

6

Som administrator ønsker jeg å analysere bruksmønstre, slik at systemet kan forbedres.

Deloppgave B (daniv)

Eksternt krav

Det er ikke spørsmål en gang om sikkerhet er et krav eller ikke, vi samler inn veldig sensitiv informasjon om brukeren og dette er så en nødvendighet å implementere for å ikke bryte norske lover (eks. GDPR).

Produktkrav

En antakelse, da vi så på Philips Hue, er at systemet kommer i en form av en app til en mobiltelefon. Av denne grunn er det viktig at systemet ikke er unødvendig stort og at ytelsen skalerer for økende enhetsliste, ved å sette at appen er 200 MB og at appen laster inn alle enheter på en budsjett-telefon innen maks. 5 sekunder.

Produktkrav

Et annet krav innen her blir oppetiden, siden funksjonaliteten viser en rapport av energiforbruk er det gunstig å ha en $\geq 95\%$ oppetid for å lagre statistikken.

Organisasjonskrav

Verifisering av "ledelsen" og sporing av tilgang til administrasjonsmodulen. Videre anonymisering av data er også ønsket. Dette er både av sikkerhetsmessige grunner og at det er uønsket at hvilken som helst som jobber på systemet kan ha innsyn i personopplysninger.

Organisasjonskrav

Bruke et programmeringsspråk kjent for utviklerene. Dette sørger for en liten læringskurve for utviklerene.

Deloppgave C (daniv)

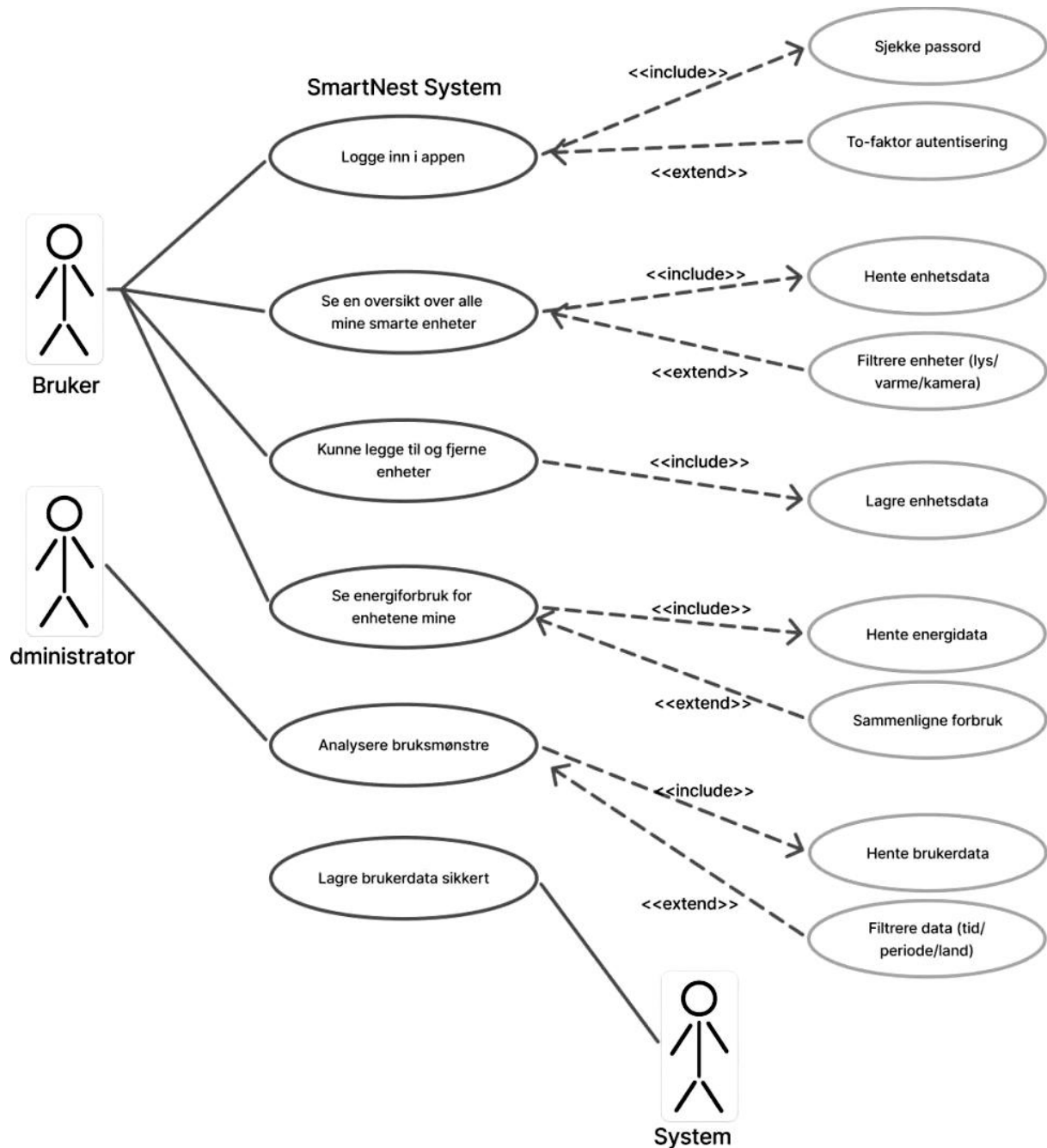
Det eksterne kravet kan verifiseres gjennom å sjekke om det stemmer med de norske lovene, gjerne ansett en spesialist til å sjekke. Her kan det som eksempel testes ved å generere en bruker som slettes i etterkant, for å se om dataene er fjernet fra systemet. Verifiser også at databasen er kryptert.

Det første produktkravet angående ytelse og størrelse kan du sjekke etter kompilasjon. Det andre produktkravet angående oppetid har blitt gitt en fast grense. Sett opp monitoring av oppetid. En kan så simulere en veldig travel dag og systemfeil på serveren og sjekke hvor stabilt systemet holder seg og om det kan restarte seg selv ved feil. Sørg for en buss faktor større eller lik 2.

Organisasjonskravet om verifisering av ledelse kan bli gjort ved å lage en bruker med manglende tillatelse og med riktig tillatelse og sjekke om kun den som har tillatelse har adgang og blir sporet av systemet når de får tilgang til administrasjonsmodulen. Det siste organisasjonskravet om programmeringsspråk må verifiseres av prosjektleder

Oppgave 5 – Use case

Deloppgave A (ninmar)



Deloppgave B (ninmar)

Use case	Aktører
Logge inn i appen	Bruker, system

Pre-betingelser:

-	Brukeren har allerede en registrert konto i SmartNest.
-	Brukeren har tilgang til internett og appen er installert.

Post-betingelser:

-	Brukeren er logget inn på sin brukerkonto.
-	Brukeren får tilgang til sitt SmartNest-dashboard.

Hovedflyt:

1	Brukeren åpner SmartNest-appen på telefonen sin.
2	Brukeren velger funksjonen ved navn "Logg inn".
3	Brukeren skriver så inn sin e-post og passord.
4	Systemet sjekker at brukernavn og passord er riktige.
5	Systemet gir tilgang til brukerens hjem-oversikt.

Alternativ flyt 1: Feil passord:

1	Hvis passordet er feil, viser systemet en feilmelding.
2	Brukeren får mulighet til å prøve innloggingen på nytt.

Alternativ flyt 2: Glemt passord:

1	Hvis man har glemt passordet, trykker man "Glemt passord".
2	Systemet sender lenke for tilbakestilling til e-post.
3	Brukeren kan lage nytt passord og prøve igjen.